Accelerating Large-Scale Decision Tree Induction on Heterogeneous CPU-GPU Architectures

Bruno Mirčevski[0009−0007−1008−6708] and Krzysztof Jurczuk[0000−0001−6469−1769]

Faculty of Computer Science, Bialystok University of Technology, Bialystok, Poland

Abstract. WIP: We investigate parallel implementations of two widely used decision tree induction algorithms, CART and C4.5, on multicore CPUs, CUDA-enabled GPUs, and heterogeneous CPU-GPU architectures.

Keywords: Decision trees · CART · C4.5 · Parallel computing

1 Introduction

Decision trees are among the most widely used models in machine learning and data mining [6]. Their predictions are easy to inspect and explain, so they suit applications in which a model must be understood or audited. They are also the base learners of random forests and gradient boosting [5,9]. CART [4] and C4.5 [15] are two of the best-known algorithms for inducing classification trees.

Growing a single tree is usually considered cheap. In practice, however, a tree is rarely trained only once. Tuning parameters such as the pruning strength, the depth limit or the minimum leaf size requires a tree for every candidate value, and k-fold cross-validation multiplies this number by k. At the same time, datasets with tens of millions of samples or thousands of features arise in areas such as particle physics [3], bioinformatics [19] and malware detection [8]. On such data, a straightforward sequential implementation can need minutes to hours per tree, and parameter tuning repeats this cost for every tree it trains. Faster induction of a single tree therefore shortens the whole modeling process.

Modern servers combine many cores on several sockets with high-bandwidth memory, and many are equipped with data-center GPUs. Using this hardware efficiently for tree induction is challenging, for two reasons. First, the work per node shrinks with depth. Nodes near the root hold most of the training set and provide enough work for thousands of GPU threads. The lower levels consist of many small nodes, for which kernel-launch, synchronization and transfer costs often dominate. Second, split evaluation does little computation per value read, so it is often limited by memory bandwidth.

Parallel decision tree induction has a long history. SLIQ [12] and SPRINT [17] introduced presorting to scale to large data. Later work targeted multicore processors [2] and distributed systems [1]. Several studies proposed GPU implementations [10,13,18], but they were evaluated on GPUs of earlier generations. Most

current high-performance GPU tree learners [5,9,16] use histogram-based split finding: continuous values are assigned to a fixed number of bins before induction. This reduces the cost, but it does not generally select the same thresholds, or produce the same trees, as an exhaustive split search under the CART or C4.5 criteria. To the best of our knowledge, no publicly available implementation grows exact CART and C4.5 trees on multicore CPUs, GPUs and both together.

In this paper, we investigate parallel implementations of CART and C4.5 for multicore CPUs, CUDA-enabled GPUs and heterogeneous CPU–GPU systems. They perform an exact split search under the CART and C4.5 criteria, without binning: every admissible cut between consecutive distinct feature values is evaluated. All implementations grow identical trees, which we checked node by node against each other and against scikit-learn for CART and Quinlan's original C4.5 program for C4.5. Performance comparisons among our implementations therefore hold the resulting tree fixed.

We evaluate five implementations, from a straightforward sequential baseline to a hybrid one that uses the CPU and the GPU together. The experiments use five public datasets with up to 32 million samples and 2,381 features, recent CPU servers, and two generations of data-center GPUs. The optimized sequential implementation is up to 25× faster than the straightforward one. Using 48 cores of a two-socket server adds up to a further 40×, and the GPU up to 4× more, for a total speed-up of up to 1,570×. Our multicore implementation trains CART trees up to 360× faster than scikit-learn [14] and C4.5 trees up to 3,100× faster than Weka [7]. On the GPU, these factors reach 1,400× and 8,100×. For deep trees, the hybrid CPU–GPU implementation is up to 4.9× faster than growing all nodes on the GPU.

The main contributions of this paper are:

- exact parallel implementations of CART and C4.5 for multicore CPUs, GPUs and heterogeneous CPU–GPU systems, with identical trees across implementations;
- a hybrid CPU–GPU scheme that assigns the nodes of a tree to devices by size, so that the CPU and the GPU grow the tree concurrently;
- an experimental evaluation on large datasets and current hardware, covering training time, memory usage, scalability, the breakdown of training time and energy consumption, including a comparison with established CPU and GPU implementations;
- an open-source release of the implementations.[1]

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 describes our implementations. Section 4 presents the experimental setup and Section 5 the results. Section 6 discusses the findings, and Section 7 concludes the paper.

[1] *[Repository URL.]*

2 Related Work

2.1 Decision Tree Induction: CART and C4.5

[Brief: CART [4] (Gini, binary splits, cost-complexity pruning), C4.5 [15] (gain ratio, MDL threshold cost, error-based pruning). Only what is needed to read Sect. 3.]

2.2 Parallel Decision Tree Induction

2.2.1 Multicore CPU

2.2.2 GPU-based

3 Our Approach

3.1 Serial Simple

3.2 Serial Optimized

3.3 Multicore CPU

3.4 GPU-Based

3.5 Hybrid

4 Experiment Design

4.1 Datasets

[One sentence per dataset (domain, how the test set is defined). All features numeric; ECBDL's nominal features as integer codes. The GPUs train ECBDL on its first 15M samples and SOREL on its first 4M (about 9.5G values each, 73 GiB of the 80 GB VRAM).]

4.2 Hardware

[Four nodes of the mordor3 cluster (Table 2), each used whole (exclusive jobs). GPU nodes run the CUDA backend only, CPU nodes the CPU backends and the other CPU implementations. GPU runs are bound to the GPU's NUMA node (16 CPUs on hopper, 24 on ampere). Add measured memory bandwidth (STREAM) per node. Software: Ubuntu 24.04, Linux 7.0, g++ 13.3 (`-O3`), CUDA 12.9, NVIDIA driver 615.71.]

Table 1. Datasets. Sizes are of the training set as float32. Every test set is the one defined with the data: the last 500,000 samples for SUSY and HIGGS (as the UCI page prescribes), the official test sets of ECBDL'14 and SOREL-20M, and for MNIST8M, which is generated from the MNIST training set, the 10,000-image MNIST test set. GPU subsamples are the first samples of the training set and use the full test set.

Dataset	Features	Classes	Training samples	Test samples	Size
SUSY [3]	18	2	4,500,000	500,000	0.3 GB
HIGGS [3]	28	2	10,500,000	500,000	1.2 GB
ECBDL'14 [19]	631	2	31,992,921	2,897,917	80.8 GB
GPU subsample	631	2	15,000,000	2,897,917	37.9 GB
MNIST8M [11]	784	10	8,100,000	10,000	25.4 GB
SOREL-20M [8]	2381	2	12,699,013	4,195,042	120.9 GB
GPU subsample	2381	2	4,000,000	4,195,042	38.1 GB

Table 2. Hardware. Every node has two sockets with SMT disabled; CPU, cores and L3 are per socket. Clock is base/max turbo in GHz. One GPU is used per run.

Node	CPU	Cores	Clock	L3	RAM	GPU
hopper	Xeon Gold 6444Y	16	3.6/4.0	45 MB	1 TB DDR5-4800	H100 80 GB
ampere	EPYC 7402	24	2.8/3.35	128 MB	1 TB DDR4-3200	A100 80 GB
epyc	EPYC 9255	24	3.25/4.3	128 MB	512 GB DDR5-6400	–
dell	Xeon Gold 6542Y	24	2.9/4.1	60 MB	1 TB DDR5-5200	–

4.3 Compared Implementations

[./tree backends (`-simple`, `-serial`, `-parallel`, `-cuda`, `-gpu-only`); scikit-learn and Weka J48 on the CPU; cuML and XGBoost on the GPU, with their settings. Why each is the closest equivalent (no public GPU C4.5; J48 depth limit adapter; XGBoost squared error = Gini on bins). rpart and YaDT: add or say why not.]

4.4 Protocol and Measurement

[One tree, depth ≤ 10, no pruning, all samples and features; what is timed (training only, from data in memory; GPU setup and upload included); peak RAM and VRAM; runs per case, means and spread; exclusive nodes; fairness (CPU threads of the hybrid, warm library processes, input layout). Source: docs/GPU-LIBRARIES.md, bench/README.md.]

5 Results

5.1 Training Time

[Overview of every implementation on every dataset (Tables 3 and 4): speed-ups of Serial Optimized over Serial Simple, Multicore CPU over Serial Optimized,

GPU over Multicore CPU, and of this work over the existing implementations. Variance: report mean with min–max, or mean ± 95 % confidence interval, once every cell has 10 runs.]

Table 3. Training time of one CART tree (depth ≤ 10, no pruning), in seconds: mean, and below it the fastest–slowest run. † marks a rough estimate (no run yet) with a plausible range below it; – not possible (data larger than the GPU's 80 GB, or no such algorithm). CPU runs on epyc1 (Multicore CPU 48 threads, the rest one thread), GPU runs on the H100 (Hybrid with 16 CPU threads). Placeholder: every cell will have at least 10 runs.

This work Dataset	Serial Simple	Serial Optimized	Multicore CPU	GPU-Based	Hybrid
SUSY	70.8 70.8–70.8	2.84 2.81–2.87	0.239 0.215–0.258	0.082 0.072–0.084	0.084 0.077–0.093
HIGGS	236 236–237	10.4 10.4–10.5	0.607 0.579–0.633	0.150 0.147–0.153	0.151 0.145–0.156
ECBDL'14	5 860	503 500–506	13.5 13.5–13.6	–	–
15M subsample	2 270	225 225–225	6.32 6.29–6.34	2.97 2.96–2.98	2.97 2.96–2.98
MNIST8M	1 880	234 233–235	5.92 5.90–5.94	3.57 3.54–3.57	3.56 3.54–3.57
SOREL-20M	12 500	878 	21.8 21.2–22.2	–	–
4M subsample	2 920	234 234–234	6.43 6.40–6.48	2.87 2.85–2.91	2.87 2.82–3.02

Existing implementations Dataset	scikit-learn	rpart	cuML	XGBoost
SUSY	86.5 86.2–87.1	*140*† 90–250	0.133 0.126–0.147	1.43 1.42–1.45
HIGGS	215 214–215	*350*† 200–600	0.456 0.456–0.456	3.43 3.29–3.58
ECBDL'14	*4 500*† 3 000–9 000	*7 500*† 4 000–15 000	–	–
15M subsample	2 200	*3 600*† 2 000–6 000	9.95 9.90–10.0	42.2 42.1–42.4
MNIST8M	*2 500*† 1 500–5 000	*4 000*† 2 000–8 000	*7*† 4–14	*50*† 25–100
SOREL-20M	*13 000*† 7 000–26 000	*21 000*† 10 000–50 000	–	–
4M subsample	*3 500*† 2 000–7 000	*6 000*† 3 000–15 000	*10*† 5–20	*45*† 25–90

5.2 Comparison with Other CPU Implementations

[./tree's backends against scikit-learn (CART) and Weka J48 (C4.5): time, memory, test accuracy, same trees.]

5.3 Comparison with GPU Libraries

[Against cuML and XGBoost: one tree, and per tree of a 10-tree forest; accuracy of exact versus binned trees; VRAM.]

Table 4. Training time of one C4.5 tree (depth ≤ 10, no pruning), in seconds: mean, and below it the fastest–slowest run. † marks a rough estimate (no run yet) with a plausible range below it; – not possible (data larger than the GPU's 80 GB, or no such algorithm). CPU runs on epyc1 (Multicore CPU 48 threads, the rest one thread), GPU runs on the H100 (Hybrid with 16 CPU threads). Placeholder: every cell will have at least 10 runs. cuML grows an entropy tree, its closest equivalent.

This work					
Dataset	Serial Simple	Serial Optimized	Multicore CPU	GPU-Based	Hybrid
SUSY	77.2 77.1–77.2	3.10 3.06–3.12	0.249 0.221–0.308	0.109 0.094–0.114	0.112 0.108–0.116
HIGGS	257 257–257	11.9 11.9–12.0	0.590 0.568–0.628	0.164 0.159–0.171	0.170 0.167–0.172
ECBDL'14	6 890	623	17.6 17.5–17.7	–	–
15M subsample	2 590	277 277–277	7.81 7.76–7.86	2.99 2.99–3.00	2.99 2.99–3.00
MNIST8M	1 970	209 209–209	5.64 5.60–5.67	3.71 3.69–3.72	3.68 3.66–3.69
SOREL-20M	12 900	1 130	28.8 28.6–29.0	–	–
4M subsample	3 090	288 288–288	7.85 7.74–7.97	3.88 3.81–3.92	4.30 4.25–4.61

Existing implementations			
Dataset	Weka J48	YaDT	cuML
SUSY	230 229–230	*46*† 25–80	0.128 0.119–0.139
HIGGS	830	*170*† 90–300	0.433 0.414–0.457
ECBDL'14	*50 000*† 25 000–100 000	*10 000*† 5 000–20 000	–
15M subsample	24 200	*4 800*† 2 500–9 000	9.89 9.84–9.98
MNIST8M	*14 000*† 7 000–28 000	*2 700*† 1 200–6 500	*7*† 3–13
SOREL-20M	*60 000*† 30 000–120 000	*12 000*† 5 000–30 000	–
4M subsample	*18 000*† 9 000–40 000	*3 800*† 1 500–9 000	*10*† 5–20

5.4 Performance Across Hardware

[All ./tree backends on the four nodes, per dataset: serial, one and two sockets, H100 and A100 hybrid and GPU-based. CPU against GPU; which hardware wins for which data shape (samples, features, classes).]

5.5 Scalability

[Thread scaling 1–48 on epyc1 and dell1 (memory bandwidth bound); growth with the number of samples (subsamples of one dataset).]

5.6 Profiling

[Where the time goes: presort, split search, partition, CPU–GPU transfer; GPU share of the hybrid; achieved bandwidth against peak.]

5.7 Energy Consumption

[Whole-server power while each implementation trains one tree on ECBDL'14 15M (the one dataset every implementation runs on, GPUs included). Method: power source and sampling rate (BMC/IPMI for the server, RAPL for the CPUs, `nvidia-smi` for the GPU), idle power measured before the runs and subtracted, energy = integral of power over the training time, 10 runs each. The CPU implementations run on a CPU node, the GPU ones on a GPU node, so the totals compare servers, not only processors. Discuss energy per tree: a fast run at high power can still use the least energy.]

Table 5. Energy of one tree on ECBDL'14 15M (depth $\leq$ 10, no pruning): mean training time t, mean power P of the whole server above idle, and energy $E = P \cdot t$. Placeholder: no measurements yet.

		CART			C4.5		
Implementation	Node	t (s)	P (W)	E (kJ)	t (s)	P (W)	E (kJ)
Serial Simple	epyc1	2 332	140	327	2 639	141	372
Serial Optimized	epyc1	225	147	33.2	278	147	40.8
Multicore CPU	epyc1	6.44	479	3.08	7.91	485	3.83
GPU-Based	hopper	2.99	316	0.95	3.03	317	0.96
Hybrid	hopper	3.01	387	1.16	3.02	394	1.19

6 Discussion

7 Conclusion

References

1. Abuzaid, F., Bradley, J.K., Liang, F.T., Feng, A., Yang, L., Zaharia, M., Talwalkar, A.S.: Yggdrasil: An optimized system for training deep decision trees at scale. In: Advances in Neural Information Processing Systems 29 (NIPS 2016). pp. 3817–3825 (2016)
2. Aldinucci, M., Ruggieri, S., Torquati, M.: Decision tree building on multi-core using FastFlow. Concurrency and Computation: Practice and Experience **26**(3), 800–820 (2014). https://doi.org/10.1002/cpe.3063
3. Baldi, P., Sadowski, P., Whiteson, D.: Searching for exotic particles in high-energy physics with deep learning. Nature Communications **5**, 4308 (2014)
4. Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: Classification and Regression Trees. Wadsworth, Belmont, CA (1984)
5. Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794 (2016)
6. Costa, V.G., Pedreira, C.E.: Recent advances in decision trees: an updated survey. Artificial Intelligence Review **56**(5), 4765–4800 (2023). https://doi.org/10.1007/s10462-022-10275-5
7. Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P., Witten, I.H.: The WEKA data mining software: An update. SIGKDD Explorations **11**(1), 10–18 (2009)
8. Harang, R., Rudd, E.M.: SOREL-20M: A large scale benchmark dataset for malicious PE detection (2020), arXiv:2012.07634
9. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems 30. pp. 3146–3154 (2017)
10. Lo, W.T., Chang, Y.S., Sheu, R.K., Chiu, C.C., Yuan, S.M.: CUDT: A CUDA based decision tree algorithm. The Scientific World Journal **2014**, 745640 (2014). https://doi.org/10.1155/2014/745640
11. Loosli, G., Canu, S., Bottou, L.: Training invariant support vector machines using selective sampling. In: Large Scale Kernel Machines, pp. 301–320. MIT Press (2007)
12. Mehta, M., Agrawal, R., Rissanen, J.: SLIQ: A fast scalable classifier for data mining. In: Advances in Database Technology – EDBT'96. LNCS, vol. 1057, pp. 18–32. Springer (1996)
13. Nasridinov, A., Lee, Y., Park, Y.H.: Decision tree construction on GPU: ubiquitous parallel computing approach. Computing **96**(5), 403–413 (2014). https://doi.org/10.1007/s00607-013-0343-z
14. Pedregosa, F., et al.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
15. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann, San Mateo, CA (1993)
16. Raschka, S., Patterson, J., Nolet, C.: Machine learning in Python: Main developments and technology trends in data science, machine learning, and artificial intelligence. Information **11**(4), 193 (2020)

17. Shafer, J., Agrawal, R., Mehta, M.: SPRINT: A scalable parallel classifier for data mining. In: Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB). pp. 544–555 (1996)
18. Strnad, D., Nerat, A.: Parallel construction of classification trees on a GPU. Concurrency and Computation: Practice and Experience **28**(5), 1417–1436 (2016). https://doi.org/10.1002/cpe.3660
19. Triguero, I., del Río, S., López, V., Bacardit, J., Benítez, J.M., Herrera, F.: ROSEFW-RF: The winner algorithm for the ECBDL'14 big data competition. Knowledge-Based Systems **87**, 69–79 (2015)

8 TEMPORARY: Chart Style Tests

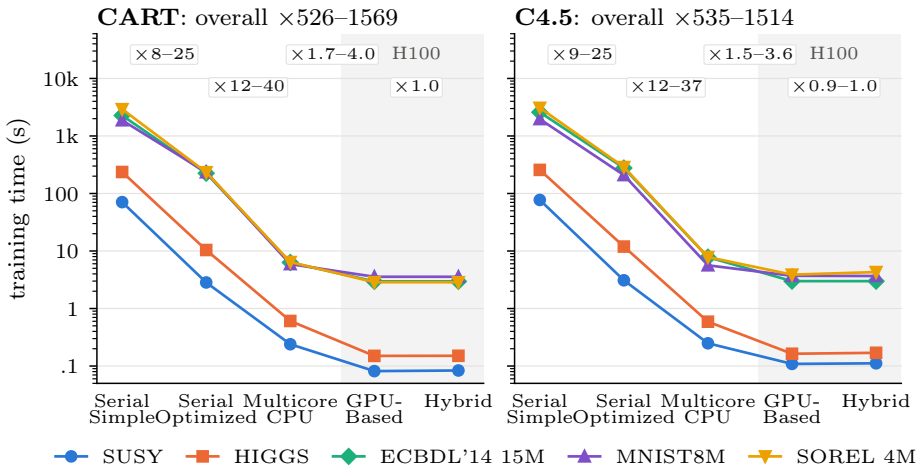


Fig. 1. Training time of one tree (depth ≤ 10, unpruned) through the steps of this work: Serial Simple, Serial Optimized and Multicore CPU on epyc1, then GPU-Based and Hybrid on the H100 (grey). Boxes: range of the step's speed-up over the five datasets.

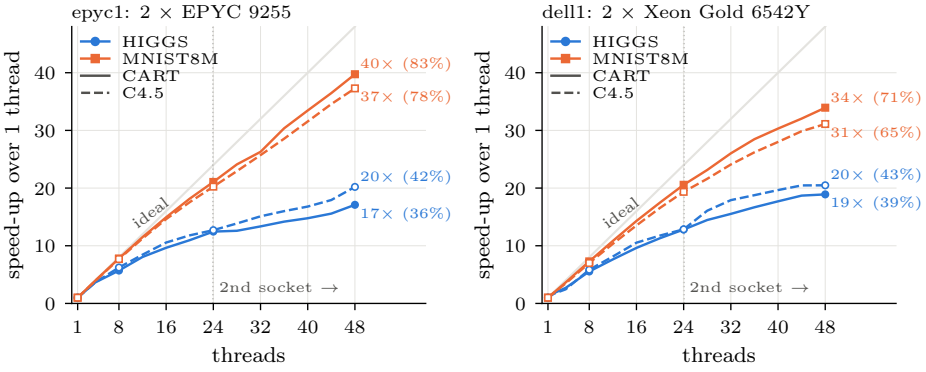


Fig. 2. Speed-up of Multicore CPU over one thread on epyc1 (left) and dell1 (right), depth $\leq$ 10, unpruned. Labels: speed-up and parallel efficiency at 48 threads; dotted line: the second socket starts at 25 threads.

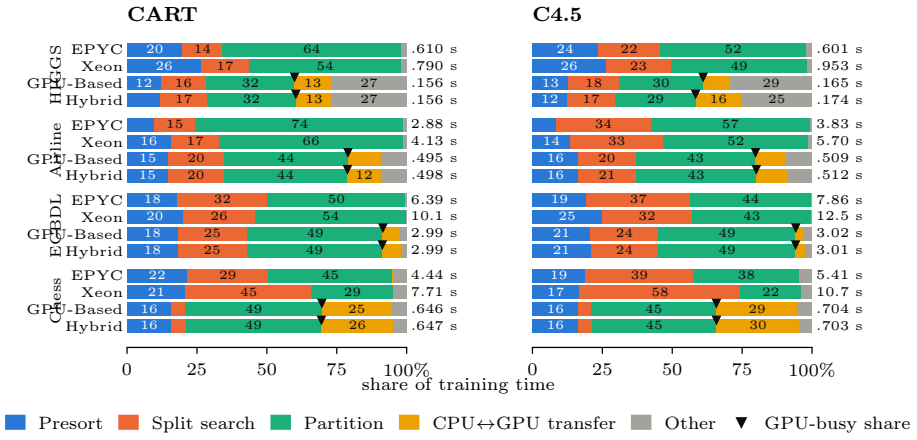


Fig. 3. Where the training time goes (percent of the training time; mean of the profiled runs, depth $\leq$ 10, unpruned). EPYC, Xeon: Multicore CPU with 48 threads on epyc1 and dell1; GPU-Based and Hybrid on the H100. Triangle: share of the time the GPU is busy. Chess: 3×3 board, 10^9 samples.

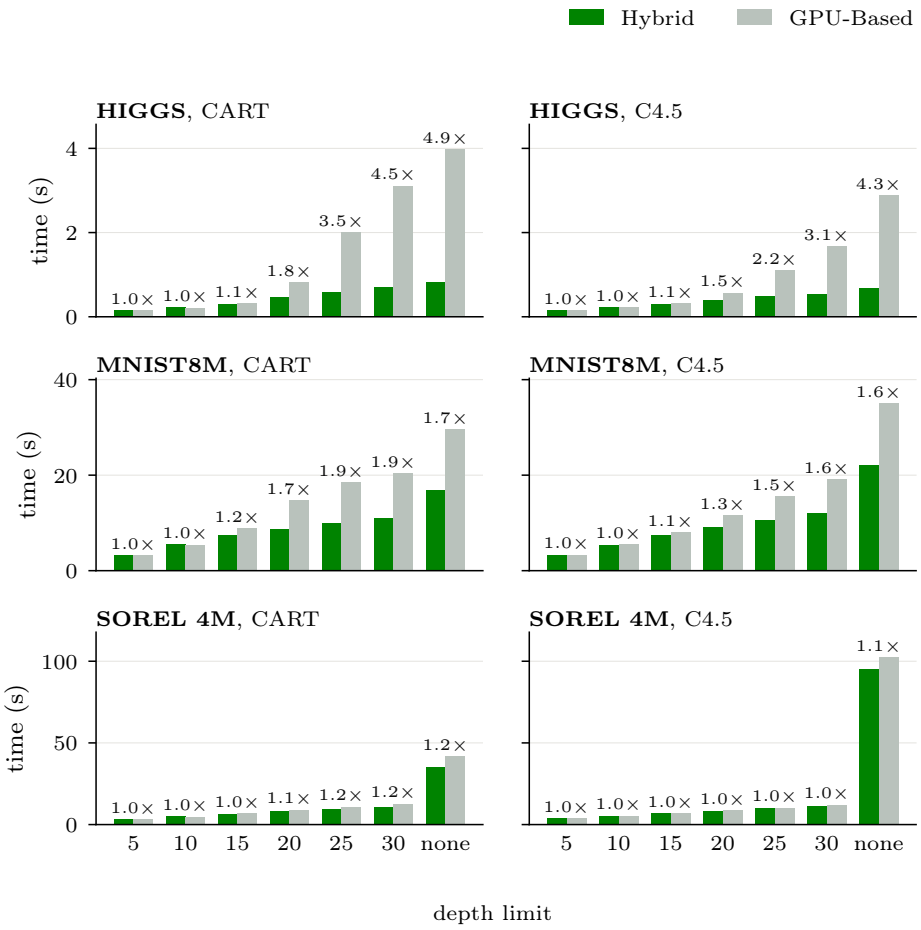


Fig. 4. Hybrid against GPU-Based by depth limit on the A100 for HIGGS, MNIST8M and the SOREL-20M 4M subsample (unpruned, mean of 3 runs, all 48 CPU threads of the node; MNIST8M at depth 5 and 10 with the 24 of the GPU's socket). Numbers above the bars: how many times slower GPU-Based is. Both modes grow the same trees.

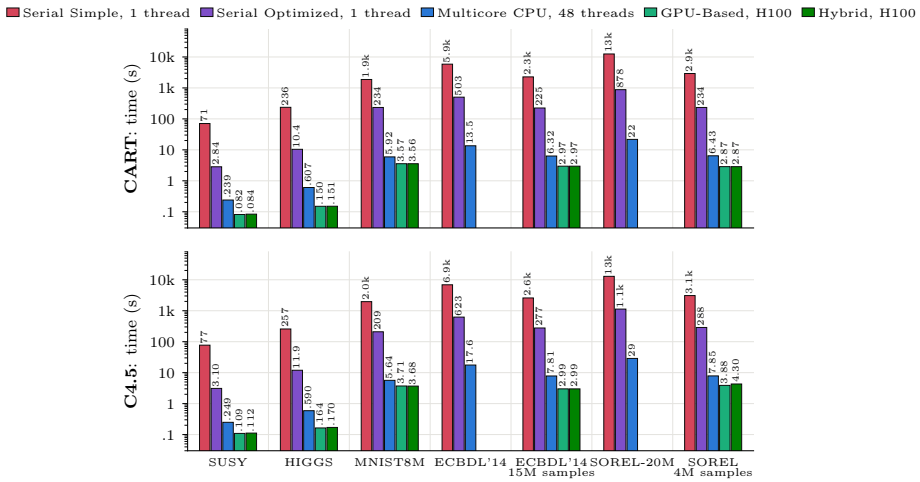


Fig. 5. Training time of one tree (depth ≤ 10, unpruned), log scale: mean over the runs. Serial Simple, Serial Optimized and Multicore CPU on epyc1, GPU-Based and Hybrid on the H100 (Hybrid with 16 CPU threads). No bar: no run (the full ECBDL'14 and SOREL-20M do not fit in the 80 GB of VRAM).

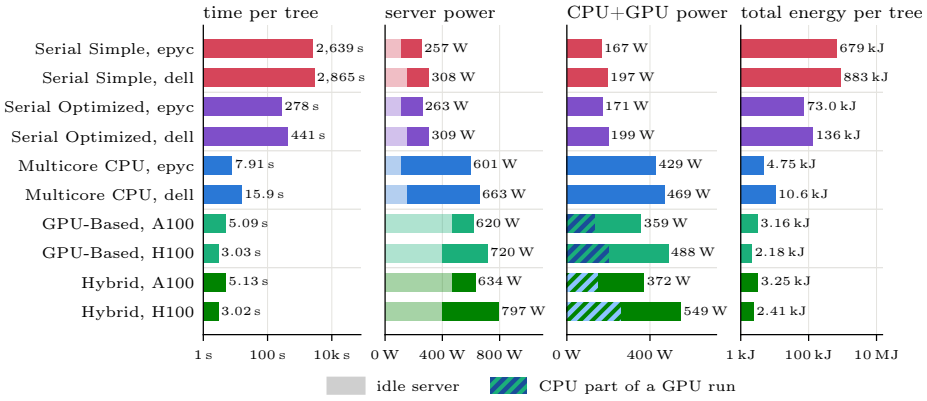


Fig. 6. Training one C4.5 tree on ECBDL'14 15M (depth ≤ 10, unpruned). Server power and energy: the whole server at the PSU, idle included. CPU+GPU power: CPU packages (RAPL) and the GPU in use (NVML). CPU implementations on epyc and dell, GPU implementations on ampere (A100) and hopper (H100).